

Exercícios — Aula 09 — Insertion sort e análise

Técnicas de Programação - 2026/02

Última atualização: September 2, 2026

Instruções

- Simule antes de escrever pseudocódigo.
- Em insertion sort, acompanhe **i**, **chave**, **j**, deslocamentos e o estado do array.
- Ao analisar custo, conte deslocamentos ou comparações no pior caso.

Exercício 1 — Simulação de insertion sort

Simule o algoritmo para $v = [10, 4, 6, 2, 3]$.

Algorithm 1: InsertionSort

Result: Executa INSERTION-SORT.

Input : array v

Output: array v ordenado no próprio lugar

```
for  $i \leftarrow 1$  to  $TAMANHO(v) - 1$  do
     $chave \leftarrow v[i]$ 
     $j \leftarrow i - 1$ 
    while  $j \geq 0$  and  $v[j] > chave$  do
         $v[j + 1] \leftarrow v[j]$ 
         $j \leftarrow j - 1$ 
    end
     $v[j + 1] \leftarrow chave$ 
end
return  $v$ 
```

Preencha uma linha para cada valor de i .

i	chave	deslocamentos feitos	posição final da chave	array ao final da iteração
1				
2				
3				
4				
5				

Qual parte do array está ordenada ao final de cada iteração?

Exercício 2 — Tabela de deslocamentos

Complete a simulação para $v = [4, 1, 3, 2]$.
Registre uma linha para cada deslocamento, não apenas para cada valor de i .

i	chave	j antes do desloca- mento	valor deslocado	array após deslocar
1				
2				
3				
3				

Depois, escreva o array final após a inserção de cada **chave**.

Exercício 3 — Ordem inversa

Considere $v = [5, 4, 3, 2, 1]$.

1. Quantos deslocamentos acontecem quando $i = 1$?
2. Quantos deslocamentos acontecem quando $i = 2$?
3. Quantos deslocamentos acontecem quando $i = 3$?
4. Quantos deslocamentos acontecem quando $i = 4$?
5. Qual é o total de deslocamentos?
6. Escreva a soma equivalente para um array de tamanho n em ordem inversa.

O custo no pior caso é constante, linear ou quadrático?

Exercício 4 — Melhor caso

Simule INSERTION-SORT para $v = [1, 2, 3, 4, 5]$.

i	chave	primeira comparação do ENQUANTO	houve deslocamento?
1			
2			
3			
4			

Responda:

1. Quantos deslocamentos acontecem no total?
2. O laço externo ainda percorre o array?
3. Por que o melhor caso é $O(n)$ e não $O(1)$?

Exercício 5 — Escrevendo o pseudocódigo

Escreva uma versão de INSERTION-SORT-COM-CONTADORES.

Contrato:

- entrada: array v ;
- saída: array ordenado, número de comparações e número de deslocamentos;
- use a mesma ideia de **chave** e deslocamento para a direita;
- conte uma comparação sempre que a condição principal entre $v[j]$ e **chave** for avaliada.

Teste sua versão com:

- $v = [1, 2, 3]$;
- $v = [3, 2, 1]$;
- $v = [2, 1, 3]$.

Exercício 6 — Comparando ordenações quadráticas

Compare insertion sort, bubble sort e selection sort nos cenários abaixo.

Cenário	Insertion sort	Bubble sort	Selection sort	Melhor escolha e justificativa
Array pequeno quase ordenado				
Array em ordem inversa				
Queremos entender trocas vizinhas				
Queremos sempre procurar o menor restante				

Use as ideias de deslocamento, troca e número de comparações.

Exercício 7 — Estabilidade

Considere os itens abaixo, em que a letra ajuda a distinguir elementos com a mesma chave numérica:

`[(4, "A"), (2, "B"), (4, "C"), (3, "D")]`

Insertion sort usa a condição `v[j].chave > chave.chave` para deslocar elementos.

1. Simule a ordenação usando apenas a chave numérica.
2. A ordem relativa de `(4, "A")` e `(4, "C")` muda?
3. O que aconteceria se a condição fosse `v[j].chave >= chave.chave`?
4. Explique o que significa dizer que uma ordenação é estável.